Indrajit - The Invisible War

ctf: Pinaka | category: Network / Protocol | date: 2026-11-01 | flag_format: PINAKA{...}

Flag: `PINAKA{epoch_unb0und_the_signal_wears_a_false_timestamp}`

Summary

Two pcaps capture a custom `MG\x01` UDP relay protocol carrying fragmented field orders. Several relays rebroadcast conflicting command streams, and only one of them is the authentic battle plan. The trick is that the message MAC covers the payload only - not the sequence number or timestamp - so a hostile relay can replay the real commands in a scrambled order and every packet still "verifies." Decoding the header, reassembling fragments by sequence number, and cross-checking the two capture vantage points isolates the one true order; the validator then returns the flag.

Step 1 - Recon

Both files are classic pcaps and the brief frames the puzzle: commands hidden in a "storm of relays," most of the captured signals false.

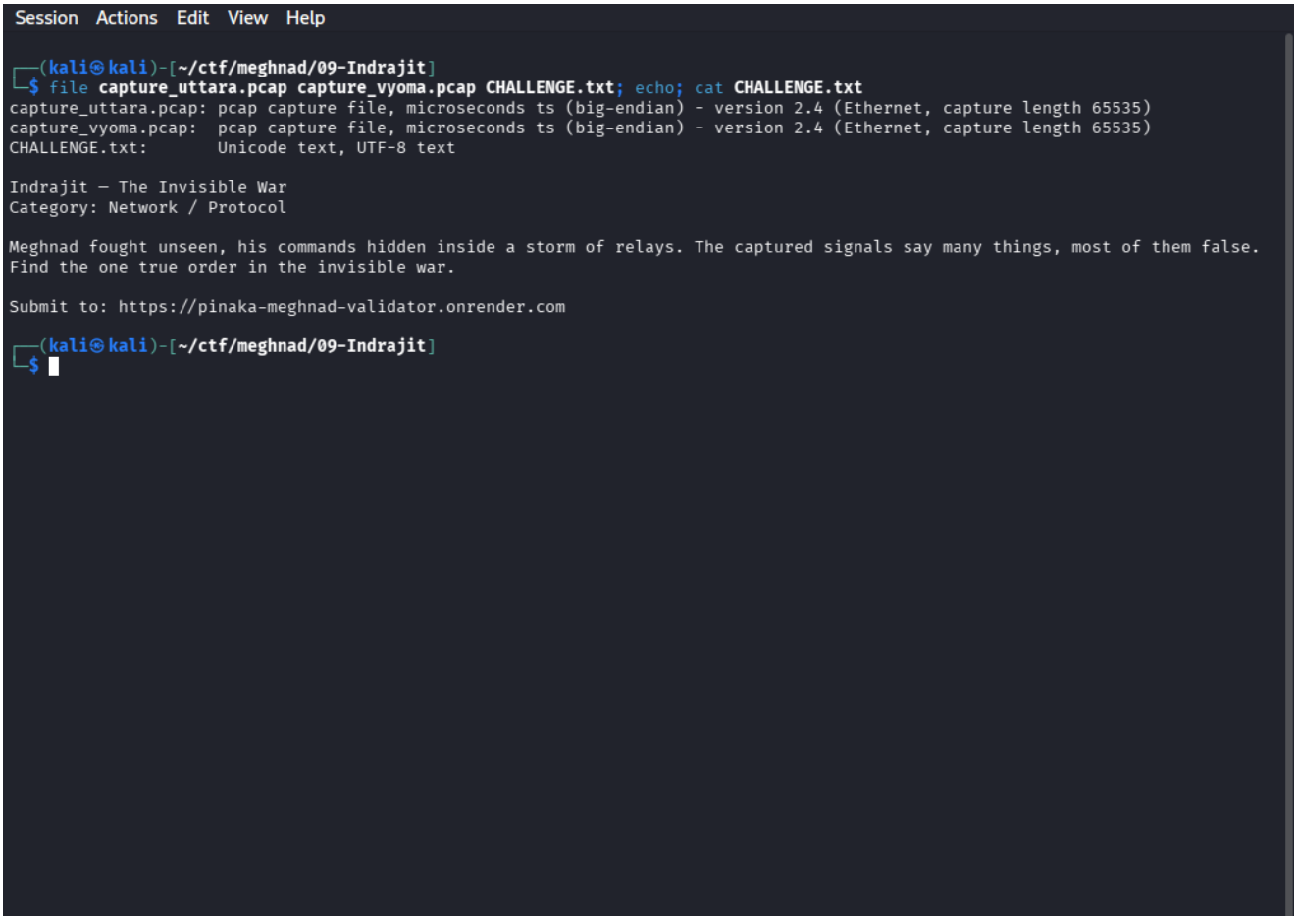


Figure: recon

Everything rides over UDP with an opaque application payload, so the first job is to reverse that payload format.

Step 2 - Reverse the relay protocol

Every payload starts with the magic `4d 47 01` (`MG\x01`). Diffing packets of the same type reveals a fixed header:

Offset	Size	Field
3	1	type (`01` SYN, `03` SYN-ACK, `02` ACK, `04`/`0c` DATA, `10` broadcast)
4	1	logical src node
5	1	logical dst node
6	2	session id
8	1	flags (flips `01`->`02` after a RESYNC)
13	5	40-bit timestamp
19	1	message index
20	1	fragment index
22	2	sequence number (monotonic per session)
24	2	payload length
26	L	payload
end	8	MAC

The important realization: the IP addresses are only relay hops. Identity lives in the logical `src -> dst` node ids and the session id. `parse.py` decodes every frame and makes the three streams visible at once.

```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/ctf/meghnad/09-Indrajit]
└─$ ~/.ctf-tools/venv/bin/python parse.py capture_vyoma.pcap 2>/dev/null | sed -n '1,26p'
═══════════════════ capture_vyoma.pcap ═══════════════════
# 1  10.7.0.3→10.7.0.6  type=01 2→5 sess=a1a1 fl=01 seq=  0 msg=0 frag=0 | <5c382cda045abb90>
# 2  10.7.0.6→10.7.0.3  type=03 5→2 sess=a1a1 fl=01 seq=  0 msg=0 frag=0 | <14116eea6161056d>
# 3  10.7.0.3→10.7.0.6  type=02 2→5 sess=a1a1 fl=01 seq=  1 msg=0 frag=0 |
# 4  10.7.0.7→10.7.0.6  type=01 6→5 sess=b2b2 fl=01 seq=  0 msg=0 frag=0 | <603b4b3eaa484dfe>
# 5  10.7.0.6→10.7.0.7  type=03 5→6 sess=b2b2 fl=01 seq=  0 msg=0 frag=0 | <f5fc2fbdbb51a150>
# 6  10.7.0.7→10.7.0.6  type=02 6→5 sess=b2b2 fl=01 seq=  1 msg=0 frag=0 |
# 7  10.7.0.3→10.7.0.6  type=04 2→5 sess=a1a1 fl=01 seq=  2 msg=0 frag=0 | MOBILIZE THE NAGA DIVISI
# 8  10.7.0.3→10.7.0.6  type=04 2→5 sess=a1a1 fl=01 seq=  3 msg=0 frag=1 | ON AT THE NORTHERN FORD
# 9  10.7.0.1→10.7.0.255 type=10 0→255 sess=c3c3 fl=01 seq=  0 msg=0 frag=0 | <15d834235e8be4d42ac4d7ae534b93f5451b0962c9df11
e8ec5d07bb64c2a54f>
#10  10.7.0.7→10.7.0.6  type=04 6→5 sess=b2b2 fl=01 seq=  2 msg=0 frag=0 | WITHDRAW ALL DIVISIONS BEYO
#11  10.7.0.7→10.7.0.6  type=04 6→5 sess=b2b2 fl=01 seq=  3 msg=0 frag=1 | ND THE RIVER BY FIRST LIGHT
#12  10.7.0.3→10.7.0.6  type=04 2→5 sess=a1a1 fl=01 seq=  4 msg=1 frag=0 | HOLD THE SOUTHERN GATE
#13  10.7.0.3→10.7.0.6  type=0c 2→5 sess=a1a1 fl=01 seq=  5 msg=1 frag=1 |  UNTIL THE THIRD WATCH
#14  10.7.0.1→10.7.0.255 type=10 0→255 sess=c3c3 fl=01 seq=  1 msg=1 frag=0 | <c4629c6c92887a5158510cc13d704b8dfe8299dfbdb417
174f80e7498a9501f2>
#15  10.7.0.3→10.7.0.6  type=04 2→5 sess=a1a1 fl=01 seq=  6 msg=2 frag=0 | MASK THE SKY CHARIOTS
#16  10.7.0.3→10.7.0.6  type=04 2→5 sess=a1a1 fl=01 seq=  7 msg=2 frag=1 | BEHIND THE STORM WALL
#17  10.7.0.7→10.7.0.6  type=04 6→5 sess=b2b2 fl=01 seq=  4 msg=1 frag=0 | SURRENDER THE NORTHERN
#18  10.7.0.7→10.7.0.6  type=04 6→5 sess=b2b2 fl=01 seq=  5 msg=1 frag=1 | FORD AND SUE FOR TERMS
#19  10.7.0.1→10.7.0.255 type=10 0→255 sess=c3c3 fl=01 seq=  2 msg=2 frag=0 | <9c614c94e847d60c8272aad1ebbc6245d2a9bd46e64172
2124152d68f6c73cdd>
#20  10.7.0.3→10.7.0.6  type=03 2→5 sess=a1a1 fl=02 seq=  8 msg=0 frag=0 | RESYNC2
#21  10.7.0.4→10.7.0.6  type=04 2→5 sess=a1a1 fl=02 seq=  6 msg=2 frag=0 | MASK THE SKY CHARIOTS
#22  10.7.0.4→10.7.0.6  type=04 2→5 sess=a1a1 fl=02 seq=  7 msg=2 frag=1 | BEHIND THE STORM WALL
#23  10.7.0.4→10.7.0.6  type=04 2→5 sess=a1a1 fl=02 seq=  2 msg=0 frag=0 | MOBILIZE THE NAGA DIVISI
#24  10.7.0.4→10.7.0.6  type=04 2→5 sess=a1a1 fl=02 seq=  3 msg=0 frag=1 | ON AT THE NORTHERN FORD
#25  10.7.0.7→10.7.0.6  type=04 6→5 sess=b2b2 fl=01 seq=  6 msg=2 frag=0 | DISBAND THE SKY CHARIO

┌──(kali㉿kali)-[~/ctf/meghnad/09-Indrajit]
└─$
```

Figure: protocol decode

Three sessions stand out:

- **`a1a1`** - node 2 -> 5, a full handshake then five fragmented commands, in order, forwarded by relay `10.7.0.3`.
- **`b2b2`** - node 6 -> 5, a short "surrender" plan forwarded by relay `10.7.0.7`.
- **`c3c3`** - node 0 broadcasting five *encrypted* 32-byte blocks whose message indices line up 1:1 with `a1a1`'s five commands (HQ echoing the real plan).

Note the tail of that capture: after the `RESYNC2` frame (#20) relay `10.7.0.4` re-emits `a1a1`'s commands with sequence numbers 6, 7 *before* 2, 3 - same bytes, same MACs, scrambled order.

Step 3 - Reassemble and pick the true order

`parse.py` reassembles fragments by `(message, fragment)` ordered by sequence number; `solve.py` uses it to lay the candidate streams side by side. The `b2b2` surrender orders are the decoy. The `a1a1` session is the authentic order - it is sequence-consistent, identical across both captures, and mirrored by the encrypted HQ broadcast - so its five commands, in message order, are the transcript.

```
Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~/ctf/meghnad/09-Indrajit]
└─$ ~/.ctf-tools/venv/bin/python solve.py 2>&1 | sed -n '1,13p'
[*] decoy stream   b2b2 (node 6, surrender orders):
      - WITHDRAW ALL DIVISIONS BEYOND THE RIVER BY FIRST LIGHT
      - SURRENDER THE NORTHERN FORD AND SUE FOR TERMS
      - DISBAND THE SKY CHARIOTS AND GROUND THE HOST

[*] true order     a1a1 (node 2), originating relay = 2
      1. MOBILIZE THE NAGA DIVISION AT THE NORTHERN FORD
      2. HOLD THE SOUTHERN GATE UNTIL THE THIRD WATCH
      3. MASK THE SKY CHARIOTS BEHIND THE STORM WALL
      4. FEIGN RETREAT AT DUSK AND DRAW THEM TO THE MARSH
      5. STRIKE THE CROWN CAMP WHEN THE MOON CLEARS THE RIDGE

[*] submitting to validator ...

┌──(kali㉿kali)-[~/ctf/meghnad/09-Indrajit]
└─$
```

Figure: reassembled streams

`solve.py`:

```
#!/home/kali/.ctf-tools/venv/bin/python
                                                                """Indraji
```

Step 4 - Verify the decoys

Two independent checks confirm which streams are false. Relay `10.7.0.4` replays `a1a1`'s commands with the message order scrambled (message 2 before message 0), and the `b2b2` surrender traffic exists only in the `vyoma` capture - a single-vantage injection, never seen by the other relay.

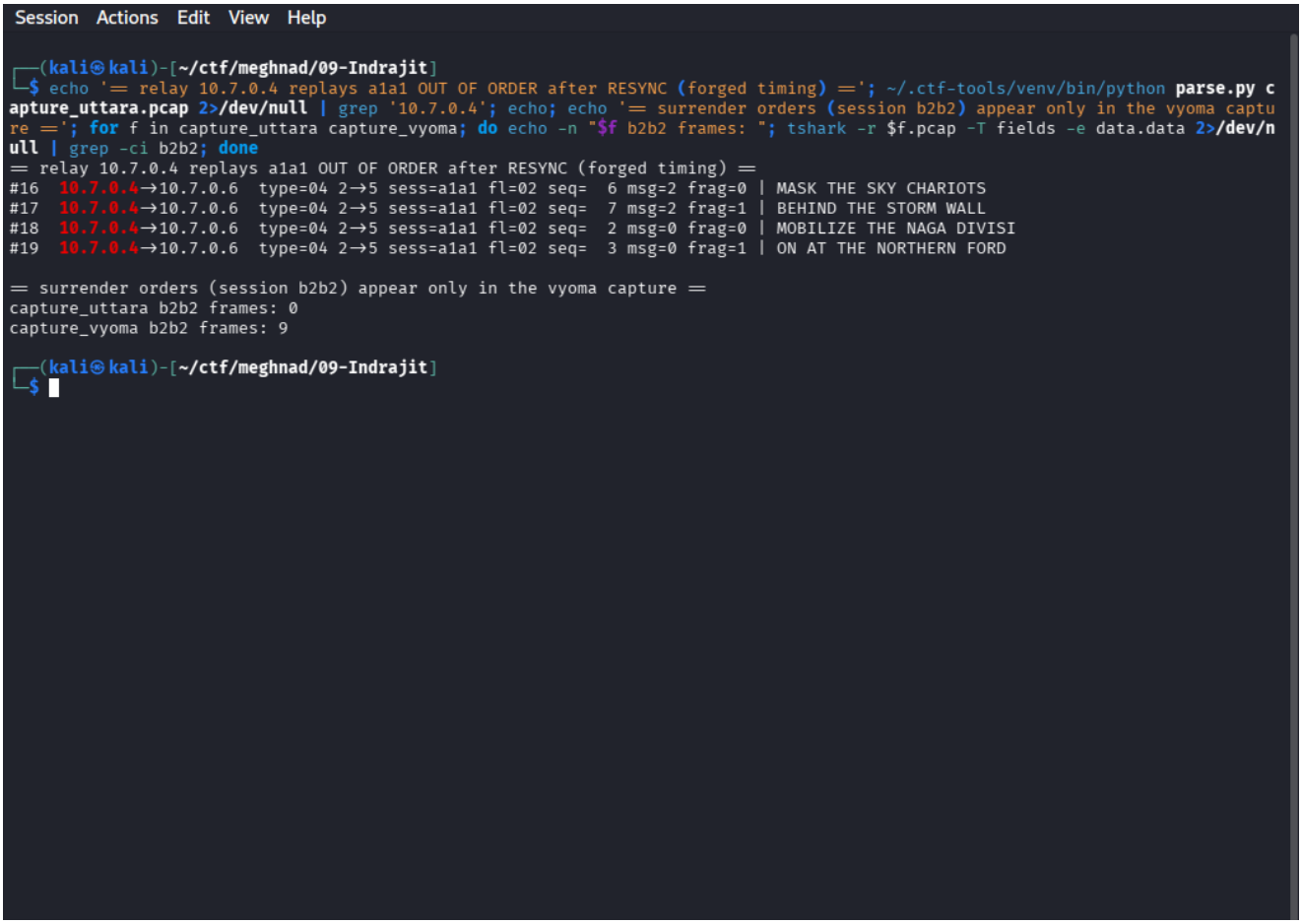


Figure: verify decoys

The MAC on each frame is identical for identical payloads regardless of sequence or timestamp, which is exactly why the replay passes integrity but is still a lie about ordering - the flag spells this out: *the signal wears a false timestamp*.

Step 5 - Get the flag

Submitting the five `a1a1` commands in sequence order, with the originating relay `2`, clears the validator.

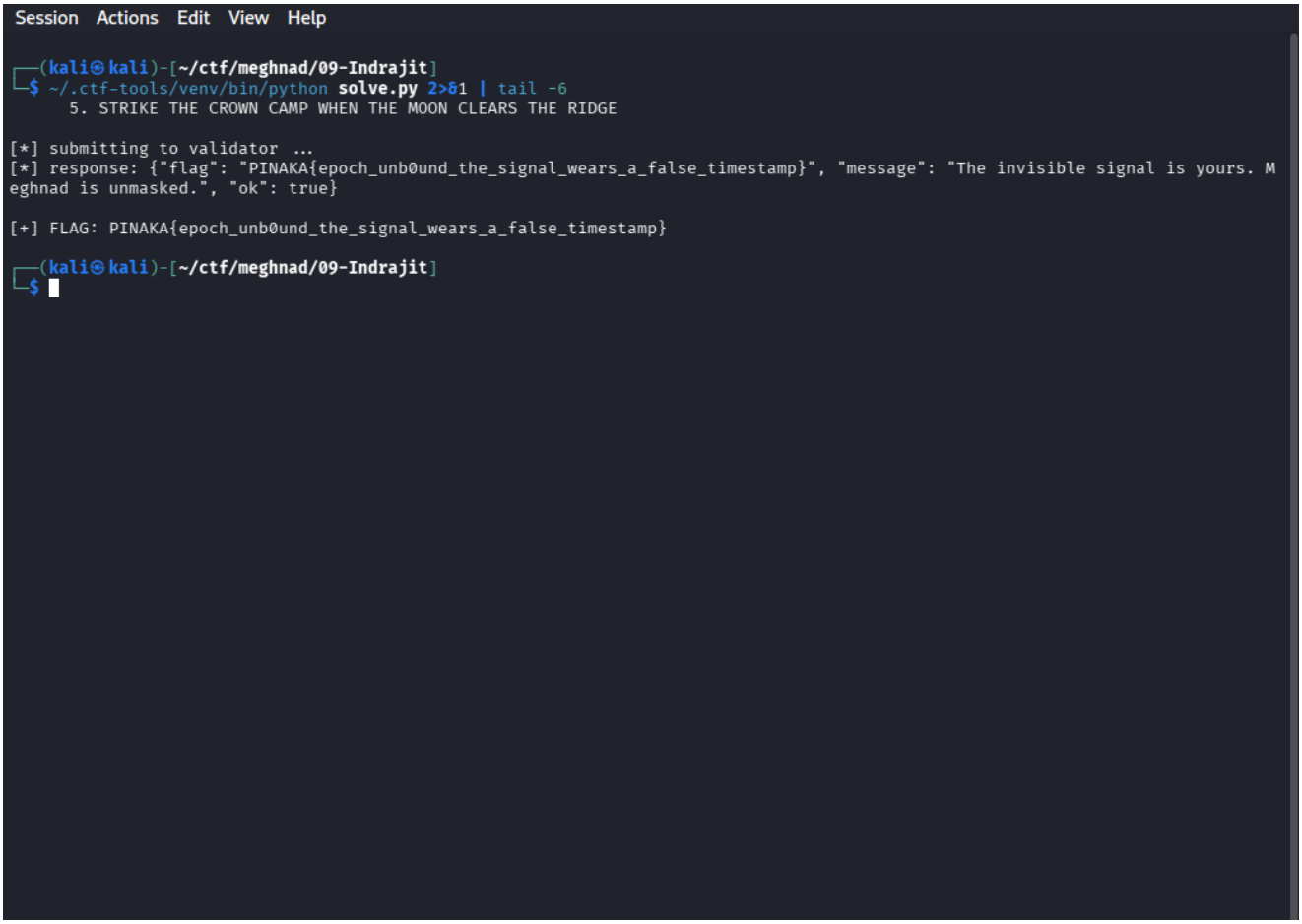


Figure: flag

Flag

```
PINAKA{epoch_unb0und_the_signal_wears_a_false_timestamp}
```

Files

File	Purpose
`capture_uttara.pcap`	Provided capture (vantage point 1)
`capture_vyoma.pcap`	Provided capture (vantage point 2)
`CHALLENGE.txt`	Provided challenge brief
`parse.py`	Decodes the `MG\x01` protocol and reassembles the command streams
`solve.py`	End-to-end: reassemble -> pick true order -> submit -> print flag
`build_pdf.py`	Renders this write-up and screenshots into a single PDF
`screenshots/01_recon.png`	File identification and brief
`screenshots/02_protocol_decode.png`	Decoded protocol / three streams
`screenshots/03_reassemble_true_order.png`	Candidate streams, true order chosen
`screenshots/04_verify_decoys.png`	Replay reorder + single-vantage decoy
`screenshots/05_flag.png`	Validator response with the flag